UNIVERSITAS MULTI DATA PALEMBANG
Fakultas Ilmu Komputer dan Rekayasa
Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, *Controller*, *Model*, dan *Middleware* pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	FX Paber Bambang Silalahi
NPM	2428240123
Kelas	SI5C
Topik Tugas 1	Daftar Paket Wisata
Entitas utama	Tour
Tautan repositori	https://github.com/pabersilalahi02-hub/si5c_kuis1_fxpaberbambangsilalahi
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya `[sumber-daya]` dan `[Entitas]`, dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, `[sumber-daya]` menjadi `buku`.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan `POST`. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama `PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf`, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian *endpoint*	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau `curl` di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan `Win` + `Shift` + `S`; macOS tekan `Cmd` + `Shift` + `4`; Linux gunakan `PrtSc` atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda `[...]` pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Daftar Paket Wisata
Entitas utama (tunggal)	Tour
Alamat dasar	/api/tours
Field data	id, name, destination, price, duration, quota
Field wajib saat POST	name, destination, price, duration, quota
Query string untuk filter	destination, maxPrice
Data awal	Paket Wisata Bali Exotis (ID 1) & Explore Jogja Istimewa (ID 2)
Modul lain (jika ada)	cors, dotenv, express

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi `cekApiKey`.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

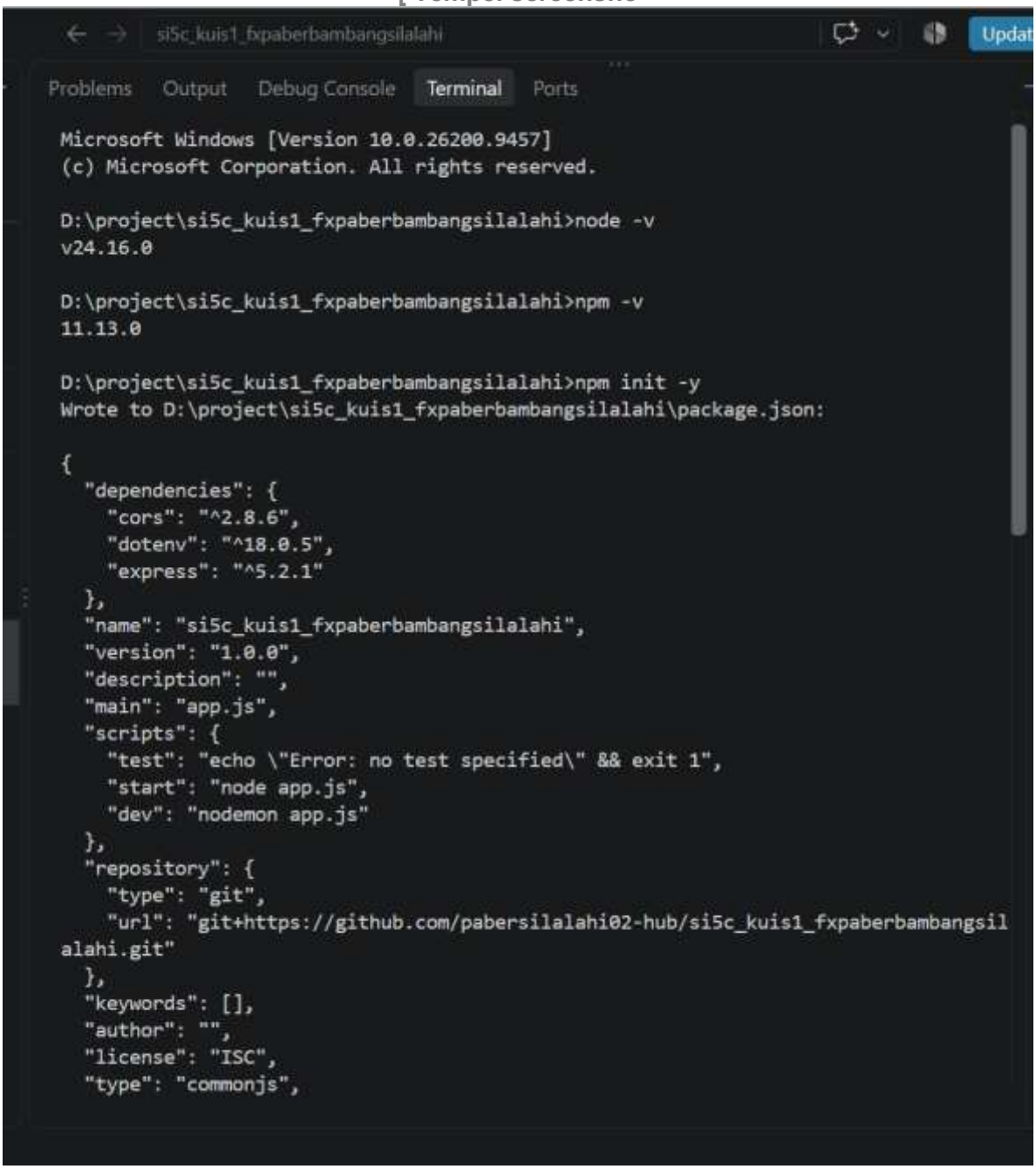
Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

[Tempel screensho



t di sini]

Gambar 1. Versi Node.js dan instalasi paket

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

Proses yang terjadi meliputi pengecekan versi Node.js (v24.16.0) dan NPM (11.13.0), diikuti perintah npm init -y untuk membuat file package.json. Berkas ini mencatat konfigurasi penting seperti daftar dependensi (express, dotenv, cors), titik masuk aplikasi (app.js), skrip eksekusi (npm start dan npm run dev), serta tautan repositori Git proyek.

Bab 2 Struktur Folder dan Ringkasan Kode

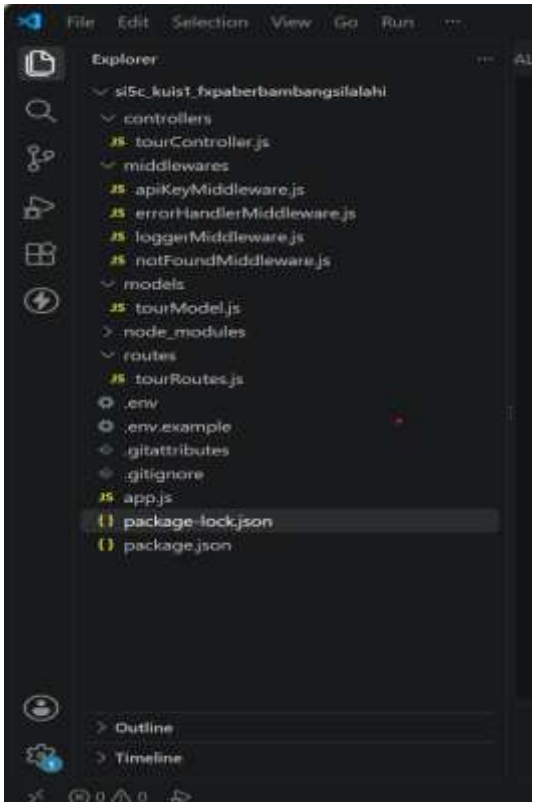
Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder `routes`, `controllers`, `models`, dan `middlewares` beserta isinya. Gunakan panel *Explorer* editor atau perintah `tree`.

[Tempel screenshot di sini]

Gambar 2. Struktur folder project



Gambar 2. Struktur folder project

Penjelasan:

JStruktur folder proyek ini menerapkan pola pada Express.js, di mana controllers mengolah logika bisnis, middlewares menangani proses perantara seperti keamanan dan log, models mengelola data paket wisata (CRUD), dan routes memetakan URL ke fungsi controller yang sesuai. Seluruh komponen ini dihubungkan melalui berkas utama app.js sebagai entry point server, dengan dukungan node_modules untuk dependensi pustaka, berkas .env untuk menyimpan variabel rahasia seperti PORT dan API_KEY, serta berkas package.json yang menyimpan konfigurasi proyek dan skrip eksekusi.

2.2 Ringkasan tugas setiap berkas

Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

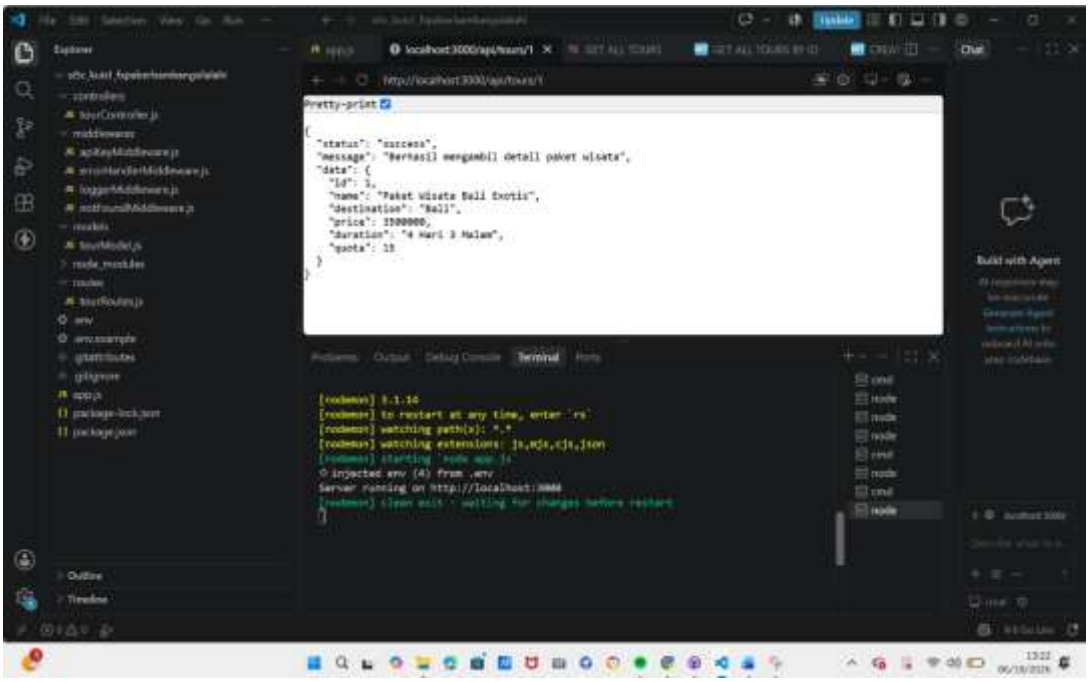
Berkas	Penjelasan Anda
models/[sumber-daya]Model.js	Berisi struktur data dan penanganan manipulasi data (CRUD). Berhubungan langsung dengan penyimpanan atau skema data paket wisata
controllers/[sumber-daya]Controller.js	Berisi logika bisnis aplikasi. Memproses permintaan dari *route*, berinteraksi dengan *model*, lalu memberikan respon HTTP ke *client*.
routes/[sumber-daya]Routes.js	Berisi pemetaan endpoint/URL API ke fungsi *controller* yang sesuai berdasarkan metode HTTP (GET, POST, PUT, DELETE)
middlewares/logger.js	yang mencatat *log* riwayat *request* masuk (seperti metode HTTP dan URL),
middlewares/cekApiKey.js	yang bertugas memvalidasi *header* API key untuk memproteksi *endpoint* sensitif,
middlewares/errorHandler.js	yang menangkap seluruh *error internal* secara global agar server tetap stabil tanpa mengalami *crash*.
app.js	Berkas utama (entry point) aplikasi Express.js yang menginisialisasi server, menghubungkan *middleware*, dan memanggil *routes*.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.

[Tempel screenshot di sini]



Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

*Tampilan tersebut menunjukkan pengujian endpoint **GET /api/tours/1** yang berhasil mengambil detail satu data paket wisata spesifik dengan ID 1 ("Paket Wisata Bali Exotis"). Server mengembalikan respon berformat JSON dengan kode status sukses beserta rincian objek data seperti nama paket, destinasi, harga, durasi, dan kuota..*

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

> **Catatan**
> Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".
> Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

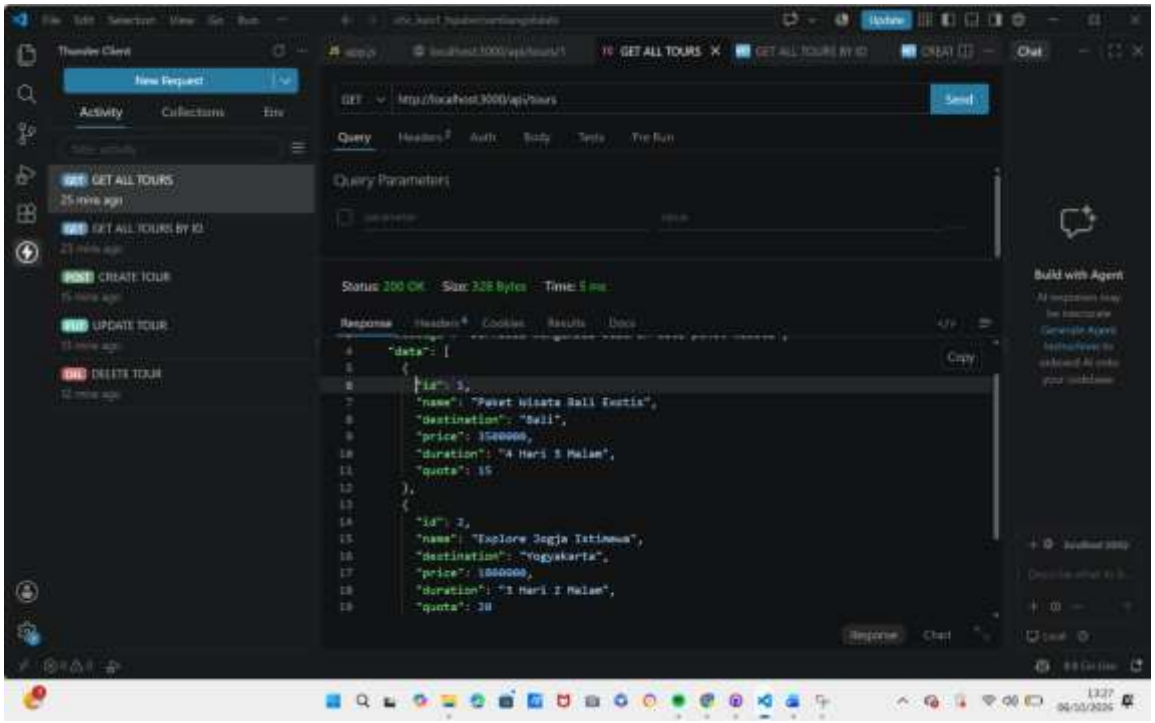
Permintaan (*request*)	GET/http://localhost:3000/api/tours
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.

Status hasil uji	{
	"status": "success",
	"message": "Berhasil mengambil seluruh data paket wisata",
	"data": [
	{
	"id": 1,
	"name": "Paket Wisata Bali Exotis",
	"destination": "Bali",
	"price": 3500000,
	"duration": "4 Hari 3 Malam",
	"quota": 15
	},
	{
	"id": 2,
	"name": "Explore Jogja Istimewa",
	"destination": "Yogyakarta",
	"price": 1800000,
	"duration": "3 Hari 2 Malam",
	"quota": 20
	}
	]
	}

Lulus?	Ya

[Tempel screenshot di sini]

Gambar 4. Hasil GET semua data



Gambar 4. Hasil GET semua data

Penjelasan:

*Request ini berhasil diproses dengan status **200 OK**, yang menandakan bahwa server sukses mengambil dan menampilkan seluruh daftar paket wisata yang tersimpan dalam format JSON, seperti paket wisata Bali dan Jogja.*

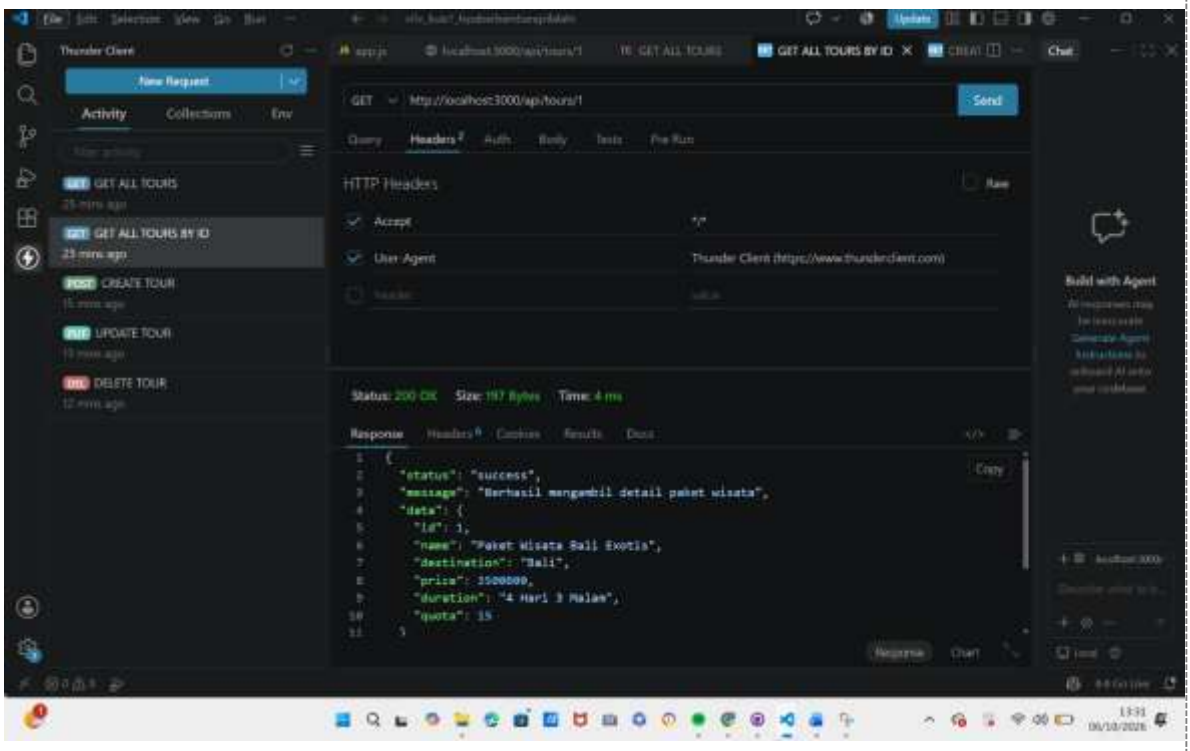
T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (*request*)	GET/http://localhost:3000/api/tours/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	{ "status": "success", "message": "Berhasil mengambil detail paket wisata", "data": { "id": 1, "name": "Paket Wisata Bali Exotis", "destination": "Bali", "price": 3500000, "duration": "4 Hari 3 Malam", "quota": 15 }
Lulus?	[ ] Ya

[Tempel screenshot di sini]

Gambar 5. Hasil GET berdasarkan ID



Gambar 5. Hasil GET berdasarkan ID

Penjelasan:

*mengambil data spesifik paket wisata berdasarkan parameter ID (1). Server merespon dengan status **200 OK** dan mengembalikan objek JSON berisi rincian lengkap paket wisata ID 1, yaitu **"Paket Wisata Bali Exotis"** beserta informasi destinasi, harga, durasi, dan kuotanya.*

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (*request*)	POST /[http://localhost:3000/api/tours
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json

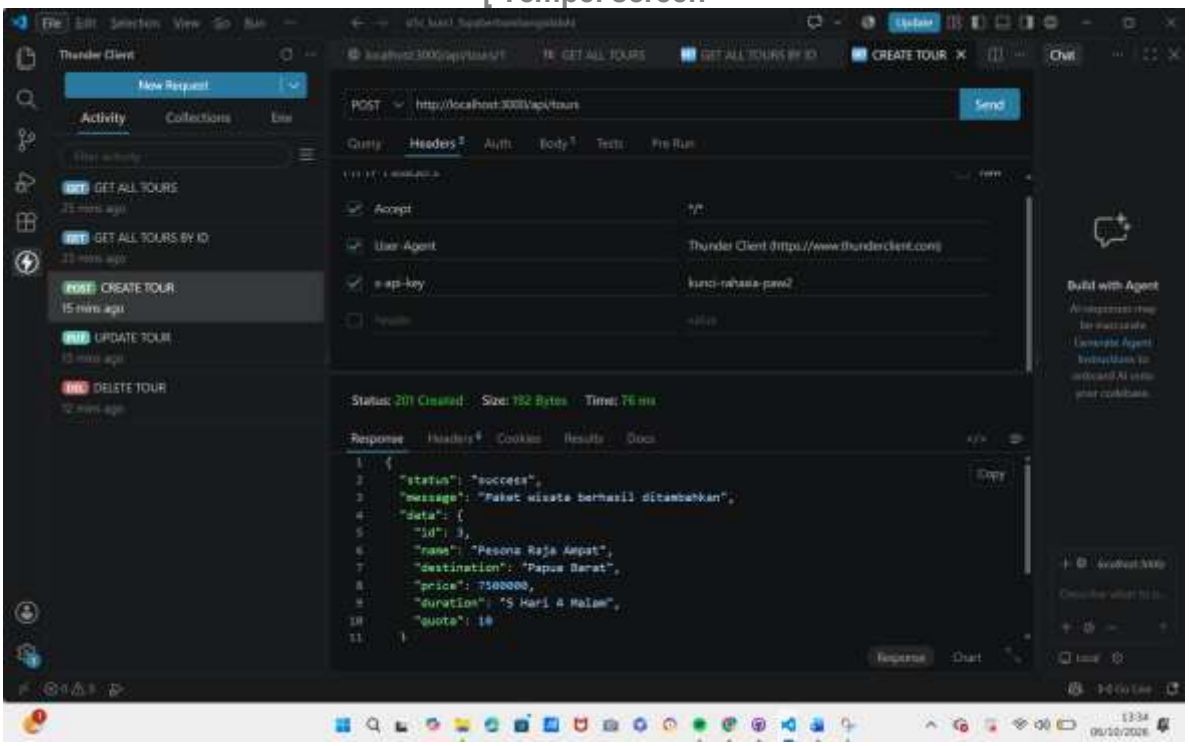
Body	{
	"name": "Pesona Raja Ampat",
	"destination": "Papua Barat",
	"price": 7500000,
	"duration": "5 Hari 4 Malam",
	"quota": 10
	}

Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.

Status hasil uji	{
	"status": "success",
	"message": "Paket wisata berhasil ditambahkan",
	"data": {
	"id": 3,
	"name": "Pesona Raja Ampat",
	"destination": "Papua Barat",
	"price": 7500000,
	"duration": "5 Hari 4 Malam",
	"quota": 10
	}
	}

Lulus?	[ ] Ya

[Tempel screen



shot di sini]

Gambar 6. Hasil POST data baru

Gambar 6. Hasil POST data baru

Penjelasan:

*Permintaan ini berhasil melewati proteksi keamanan karena menggunakan header **x-api-key: kunci-rahasia-paw2**. Server mengembalikan status **201 Created** dengan respon JSON yang mengonfirmasi bahwa data baru bertajuk **"Pesona Raja Ampat"** (ID: 3) telah sukses ditambahkan ke dalam database*

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (*request*)	PUT /http://localhost:3000/api/tours/1
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json

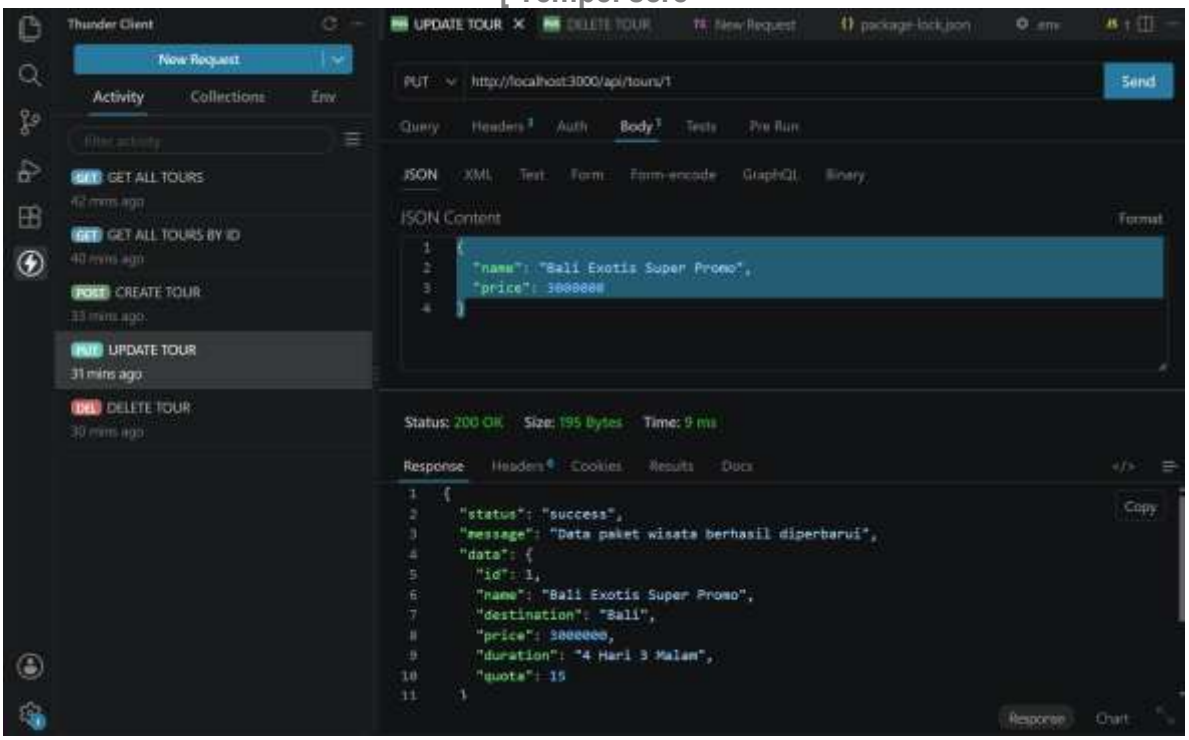
Body	{
	"name": "Bali Exotis Super Promo",
	"price": 3000000
	}

Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.

Status hasil uji	{
	"status": "success",
	"message": "Data paket wisata berhasil diperbarui",
	"data": {
	"id": 1,
	"name": "Bali Exotis Super Promo",
	"destination": "Bali",
	"price": 3000000,
	"duration": "4 Hari 3 Malam",
	"quota": 15
	}
	}

Lulus?	[ ] Ya

[Tempel scre



enshot di sini]

Gambar 7. Hasil PUT mengubah data

Gambar 7. Hasil PUT mengubah data

Penjelasan:

*Permintaan dikirim dengan membawa body JSON berupa informasi baru, yaitu "name": "Bali Exotis Super Promo" dan "price": 3000000. Server merespon dengan status **200 OK** dan mengembalikan objek data ID 1 yang nilainya telah sukses diperbarui sesuai perubahan tersebut.*

T05 Menghapus data (DELETE)

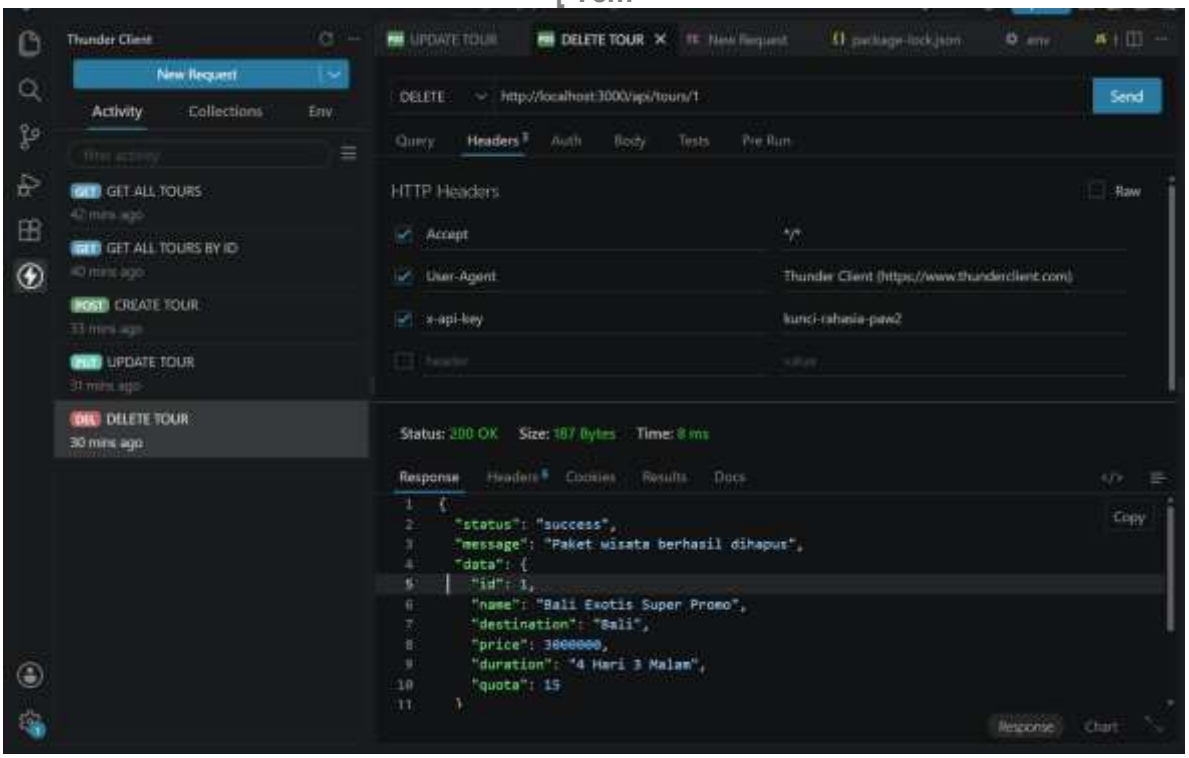
Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (*request*)	DELETE /http://localhost:3000/api/tours/1
Header	x-api-key: kunci-rahasia-paw2
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.

Status hasil uji	{
	"status": "success",
	"message": "Paket wisata berhasil dihapus",
	"data": {
	"id": 1,
	"name": "Bali Exotis Super Promo",
	"destination": "Bali",
	"price": 3000000,
	"duration": "4 Hari 3 Malam",
	"quota": 15
	}
	}

Lulus?	[ ] Ya

[Tem



pel screenshot di sini]

Gambar 8. Hasil DELETE data

Gambar 8. Hasil DELETE data

Penjelasan:

*Permintaan ini berhasil diproses karena mengirimkan header **x-api-key: kunci-rahasia-paw2** sebagai otentikasi. Server merespon dengan status **200 OK** dan menampilkan pesan "Paket wisata berhasil dihapus" beserta data paket wisata ID 1 yang telah terhapus dari sistem.*

Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (*request*)	POST /[http://localhost:3000/api/tours (tanpa x-api-key)
Header	Content-Type: application/json

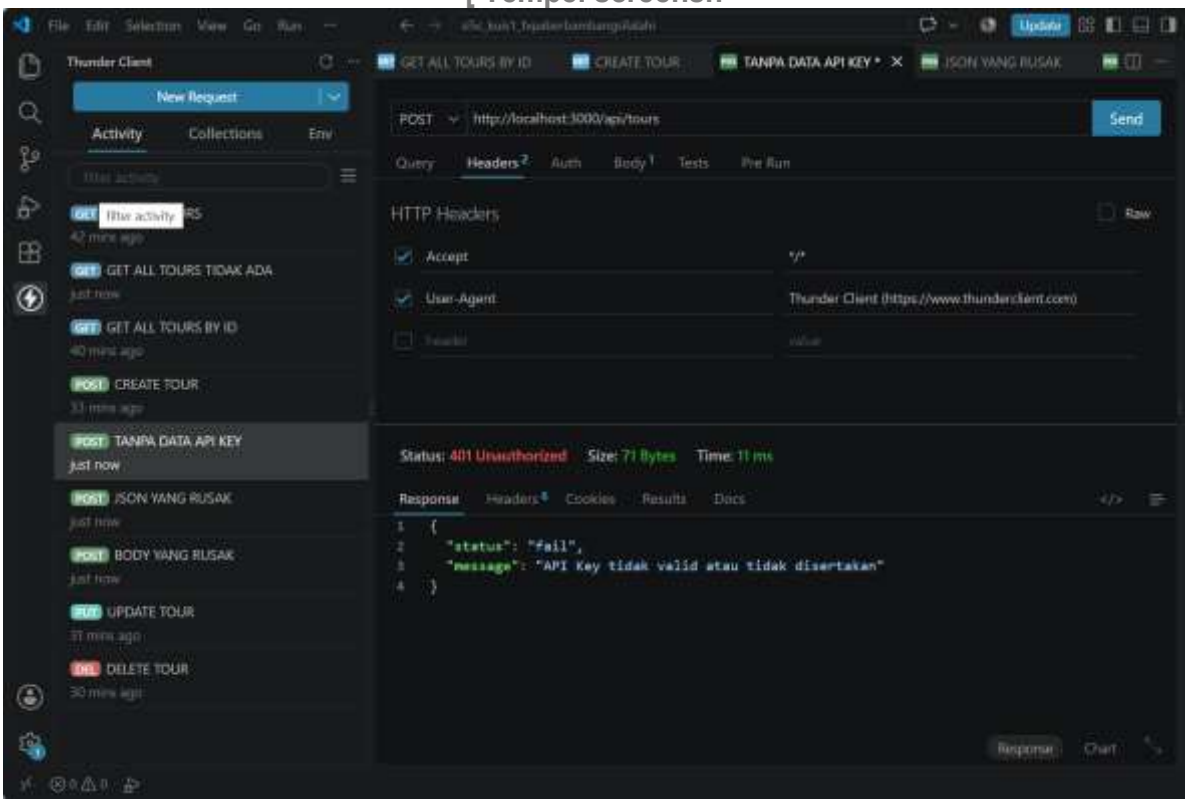
Body	{
	"name": "Pesona Raja Ampat",
	"destination": "Papua Barat",
	"price": 7500000,
	"duration": "5 Hari 4 Malam",
	"quota": 10
	}

Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.

Status hasil uji	{
	"status": "fail",
	"message": "API Key tidak valid atau tidak disertakan"
	}

Lulus?	[ ] Ya

[Tempel screensh



ot di sini]

Gambar 9. POST tanpa API key ditolak

Gambar 9. POST tanpa API key ditolak

Penjelasan:

*Permintaan ini dikirim tanpa menyertakan header x-api-key. Sistem melalui apiKeyMiddleware.js menolak akses tersebut dan mengembalikan respon status **401 Unauthorized** dengan pesan JSON "API Key tidak valid atau tidak disertakan". Ini membuktikan bahwa mekanisme otentikasi perantara (middleware) berhasil memproteksi endpoint sensitif dari akses yang tidak terotorisasi.*

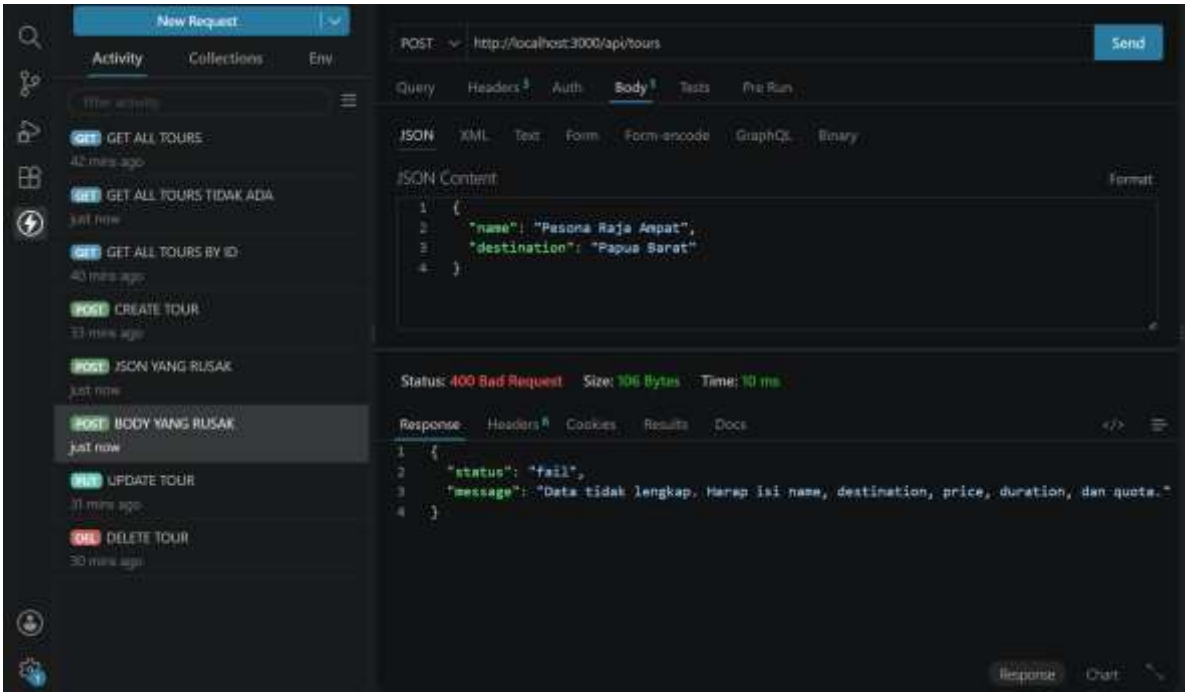
N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (*request*)	POST/ http://localhost:3000/api/tours
Header	x-api-key: (nilai API_KEY pada .env Anda) Content-Type: application/json
Body	{ "name": "Pesona Raja Ampat", "destination": "Papua Barat" }
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.

Status hasil uji	{
	"status": "fail",
	"message": "Data tidak lengkap. Harap isi name, destination, price, duration, dan quota."
	}

Lulus?	[ ] Ya



0060[Tempel screenshot di sini]

Gambar 10. POST dengan body tidak lengkap

Gambar 10. POST dengan body tidak lengkap

Penjelasan:

*Permintaan ini hanya mengirimkan atribut "name" dan "destination", sehingga mengabaikan atribut wajib lainnya seperti price, duration, dan quota. Sistem memvalidasi masukan tersebut dan mengembalikan respon status **400 Bad Request** dengan pesan*

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (*request*)	POST /http://localhost:3000/api/tours
Header	x-api-key: kunci-rahasia-paw2

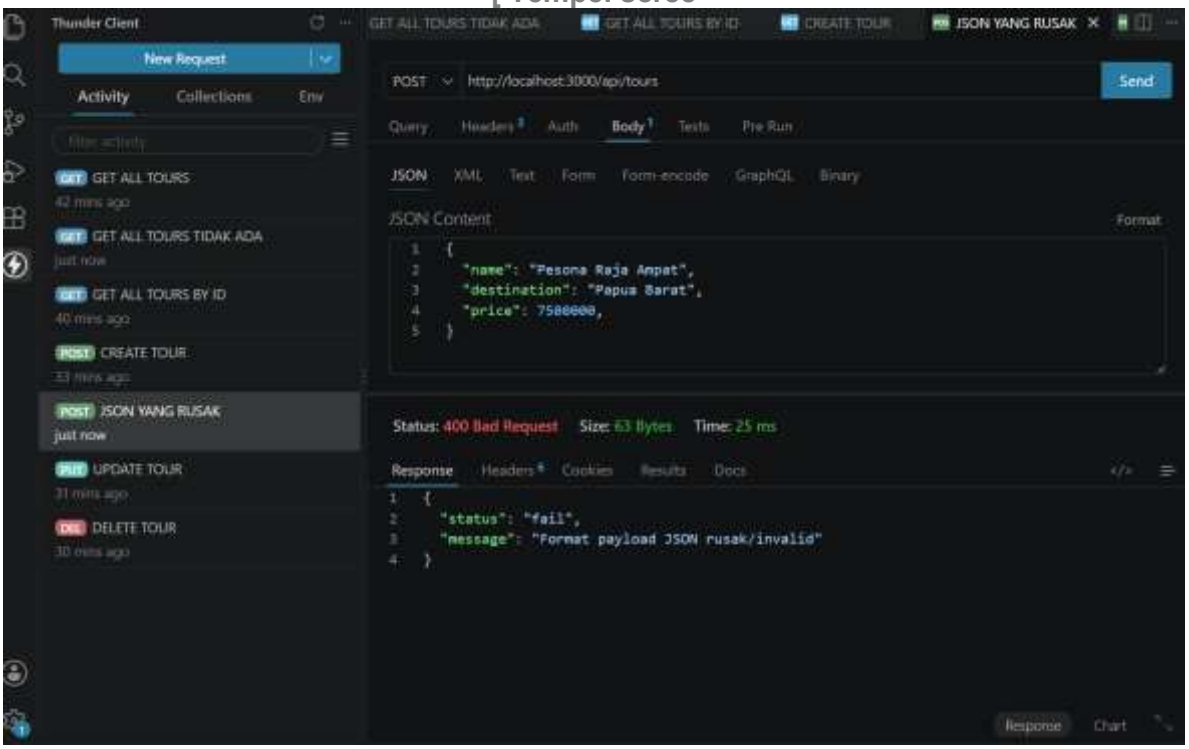
Body	{
	"name": "Pesona Raja Ampat",
	"destination": "Papua Barat",
	"price": 7500000,
	}

Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).

Status hasil uji	{
	"status": "fail",
	"message": "Format payload JSON rusak/invalid"
	}

Lulus?	[ ] Ya

[Tempel scree



nshot di sini]

Gambar 11. POST dengan JSON rusak

Gambar 11. POST dengan JSON rusak

Penjelasan:

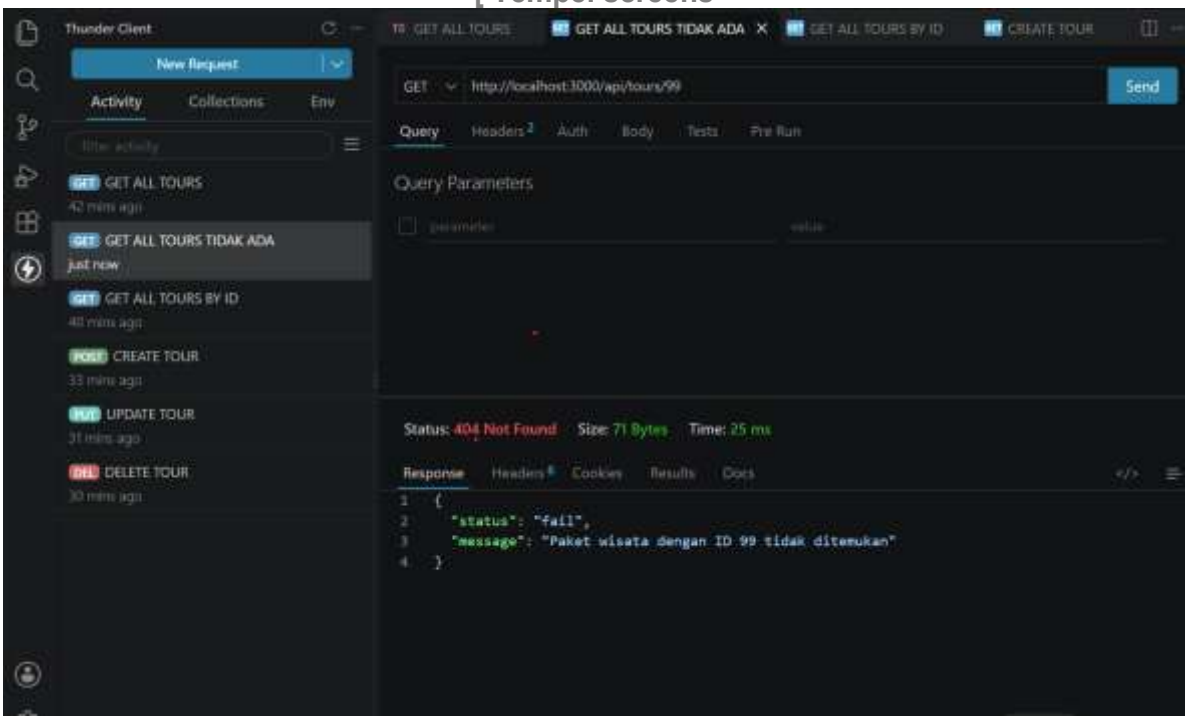
*Permintaan ini menghasilkan respon status **404 Not Found** dengan pesan JSON "Paket wisata dengan ID 99 tidak ditemukan". Ini membuktikan bahwa sistem validasi ID pada controller berfungsi dengan baik saat menangani data yang tidak terdaftar.*

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (*request*)	GET /http://localhost:3000/api/tours/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	(isi dengan kode status dan pesan yang tampil pada screenshot Anda)
Lulus?	[ ] Ya

[Tempel screens



hot di sini]

Gambar 12. GET ID tidak ada

Gambar 12. GET ID tidak ada

Penjelasan:

*Permintaan ini menghasilkan respon status **404 Not Found** dengan pesan JSON "Paket wisata dengan ID 99 tidak ditemukan". Ini membuktikan bahwa sistem validasi ID pada controller berfungsi dengan baik saat menangani data yang tidak terdaftar.*

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200		Lulus
T02	Menampilkan satu data berdasarkan ID	200		Lulus
T03	Menambah data baru	201		Lulus
T04	Mengubah data	200		Lulus
T05	Menghapus data	204		Lulus
N01	Menambah data tanpa API key	401		Lulus
N02	Menambah data dengan body tidak lengkap	400		Lulus
N03	Mengirim JSON yang rusak	400		Lulus
N04	Mengambil ID yang tidak ada	404		Lulus

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/pabersilalahi02-hub/si5c_kuis1_fxpaberbambangsilalahi
Pesan commit terakhir	Kuis 1
Folder routes, controllers, models, middlewares ada	[ ] Ya
.env tidak terunggah (ada di .gitignore)	[ ] Ya
.env.example ada	[ ] Ya

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:
Tulis jawaban Anda di sini.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:
Tulis jawaban Anda di sini.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda `[...]` sudah diganti dengan topik Tugas 1 saya	YA
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	YA
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	YA
4	Identitas saya (nama atau NIM) muncul pada data uji POST	YA
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	YA
6	Tabel rekapitulasi sesuai dengan tangkapan layar	YA
7	Tautan repositori pada sampul dapat dibuka dan `.env` tidak terunggah	YA
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	YA

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; `.env` tidak terunggah	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		**100**